

	Module 1	Marks
	Chapter 1- Introduction	
1.	Explain Language Processing System With Neat Diagram In Detail	8M
2.	Explain With a Neat Diagram The Phases Of A Compiler, Mention The Input And Output Of Each Phase Of Compiler With An Example Position:= Initial + Rate * 60	12M
3.	Explain The Structure Of The Compiler With Neat Diagram.	4M
4.	Briefly Explain The Compiler Construction Tools	3M
5.	Briefly Explain About Cross Compiler And Bootstrapping With An Example.	6M
	Chapter 2- Lexical Analysis	
6.	With Neat Diagram Explain The Role Of Lexical Analyzer and Interaction Of Lexical Analyzer And The Parser	5M
7.	Explain Input Buffering Strategy Used in Lexical Analysis With a Neat Diagram And Also write An Algorithm For Look Ahead Code With Sentinels	8M
8.	Define the Following I. Token II. Pattern III. Lexemes IV. Regular Expression V. Sentinel	6M
9.	Write Regular Expression And Construct Transition Diagram For The Following Along With An Implementation (Pseudo Code). I. Relational Operator Ii. Identifier Iii. Keyword Iv. Unsigned Number V. White Space	5*6=30M
10.	Explain The Lex Tool Along With Its Structure And Example(Program)	10M
	MODULE 2 SYNTAX ANALYSIS I	
11.	With Neat Diagram Briefly Explain The Role Of Parser And Its Types.	5M
12.	Give The Formal Definition Of Context-Free Grammars With An Example	4M
13.	Explain different types of error recovery strategies with an example.	6M
14.	Define left most derivation and rightmost derivation with example	4M
15.	Explain elimination of ambiguity and dangling else problem for the following example. Stmt → if expr then stmt if expr then stmt else stmt other	8M
16.	Define left recursion and left factoring with an example	4M
17.	Eliminate left recursion for the following grammar I) E → E + T T T → T * F F F → (E) ID	4*3=12M

	II) $S \rightarrow Aa b$ $A \rightarrow Ac Sd \epsilon$ III) $A \rightarrow Ba Aa c$ $B \rightarrow Bb Ab d$ IV) $\text{exp} \rightarrow \text{exp addop term term}$ $\text{term} \rightarrow \text{term mulop factor factor}$	
18.	Eliminate the left factoring for the following grammar I) $S \rightarrow iEtS iEtSeS a$ $E \rightarrow b$ II) $\text{expr} \rightarrow \text{expr} + \text{term term}$ $\text{term} \rightarrow \text{term factor factor}$ III) $\text{lexp_seq} \rightarrow \text{lexp,lexp_seq lexp}$ $\text{exp} \rightarrow \text{atom list}$ $\text{atom} \rightarrow \text{number identifier}$ $\text{list} \rightarrow (\text{lexp_seq})$	3*3=9M
19.	Give the algorithm for the following i) FIRST and FOLLOW ii) Predictive parsing algorithm with diagram iii) Construction of parsing table.	12M
20.	Explain Recursive descent parser with an algorithm	
21.	Construct the predictive parsing table for $E \rightarrow E+T T$ $T \rightarrow T*F F$ $F \rightarrow (E) id$ and parse the string $id+id*id$	12M
22.	Construct the nonrecursive predictive parse table for the given grammar and check the acceptance of input string $3-(4+5*6)$ $\text{exp} \rightarrow \text{exp addop term term}$ $\text{addop} \rightarrow + -$ $\text{term} \rightarrow \text{term mulop factor factor}$ $\text{mulop} \rightarrow *$ $\text{factor} \rightarrow (\text{exp}) \text{number}$	12M
23.	Construct predictive parsing table for the following grammar and show that whether given grammar is in LL(1) or not I) $\text{stmt_seq} \rightarrow \text{stmt;stmt_seq stmt}$ $\text{stmt} \rightarrow s$ II) $S \rightarrow AaAb BbBa$ $A \rightarrow \epsilon$ $B \rightarrow \epsilon$ and the input string is ba	8MARKS EACH

	III) $S \rightarrow A$ $A \rightarrow aB Ad$ $B \rightarrow bBC f$ $C \rightarrow g$ IV) $S \rightarrow aB aC Sd Se$ $B \rightarrow bBc f$ $C \rightarrow g$ V) $lexp \rightarrow number (op\ lexp_seq)$ $op \rightarrow + - *$ $lexp_seq \rightarrow lexp_seq\ lexp lexp$ VI) $declaration \rightarrow type\ var_list$ $type \rightarrow int float$ $var_list \rightarrow identifier, var_list identifier$ show the grammar is in LL(1) and parse the string int x,y,z VII) $E \rightarrow TE'$ $E' \rightarrow +TE' \epsilon$ $T \rightarrow FT'$ $T' \rightarrow *FT' \epsilon$ $F \rightarrow (E) id$				
	MODULE 3 SYNTAX ANALYSIS II				
1.	Define The Following with an example. a) Bottom Up Parsing b) Handle c) Handle Pruning				3*1=3M
2.	CONSIDER THE GRAMMAR AND PERFORM SHIFT-REDUCE PARSING				
	$S \rightarrow TL;$ $T \rightarrow int float$ $L \rightarrow L, id id$ String int id,id;	$E \rightarrow (L)$ $L \rightarrow L, E E$ String (a,(a,a))	$E \rightarrow E+E$ $E \rightarrow E * E$ $E \rightarrow (E)$ $E \rightarrow id$ String id+id*id	$E \rightarrow E+T T$ $T \rightarrow T * F F$ $F \rightarrow (E) id$ String id*id	5 M Each
3.	Explain different types of conflicts during the shift-reduce parsing with an example.				10 M
4.	Explain the following a. Advantages of using LR parsers b. The drawback of LR grammar c. LR(0) Items d. Augmented grammar				2 M Each
5.	With an algorithm construct SLR parsing table for the following grammar				
	$E \rightarrow E+n n$ String n+n+n	$A \rightarrow (A) a$ String ((a))	$S \rightarrow (S)S \epsilon$ String ()()	$E \rightarrow E+T T$ $T \rightarrow T * F F$ $F \rightarrow (E) id$ String id*id+id	10M Each
6.	With an algorithm, Verify the grammar whether given grammar is SLR or not				
	$S \rightarrow AaAb BbBa$ $A \rightarrow \epsilon$		$S \rightarrow L=R R$ $L \rightarrow *R id$		10 Mark Each

	B->€	R->L	
7.	With an algorithm construct LR parsing table for the following grammar		
	S->CC C->Ac d String aadd	A->(A) a	10 M Each
8.	With an algorithm construct LALR parsing table for the following grammar S->CC C->Ac d		10 M
9.	Show that following grammar is LALR(1) not SLR(1) S->Aa bAc dc bda A->d		10M
10.	Explain the operator precedence parsing with an example.		10 M
11.	Explain parser generators in detail, along with the syntax of YACC execution.		6M
12.	Define the following with an example. a. SDD b. Synthesized attribute c. Inherited attribute d. Annotated parse tree e. Annotating		10M
	MODULE 4 SYNTAX-DIRECTED TRANSLATION		
13.	Consider the following grammar, construct the SDD, and annotated parse		
	S->EN E->E+T E->E-T E->T T->T*F T->T/F T->F F->(E) F->digit N->; String 5*6+7;	L->En E->E+T E->T T->T*F T->F F->(E) F->digit String 3*5+4n	10 M EACH
	T->FT' T'->*FT' T'->€ F->digit String 3*5	D->TL T->int T->float L->L,id L->id String float id ₁ ,id ₂ ,id ₃	10 M EACH
14.	Define the dependency graph with an example.		3M
15.	Design the dependency graph for the declaration Statement float id₁,id₂,id₃ and 3*2		10M
16.	Explain S attributes and L attribute definition with an example.		6 M
17.	Explain the construction of the syntax tree along with the functions used for expressions with an example.		6M
18.	Construct the syntax tree for the expression x*y-5+z along with a sequence of functions.		5M
19.	Obtain the SDD for the following expression and also obtain the syntax tree/annotated parse tree for the expression a-4+c for the grammar E->E+T E->E-T E->T		10M

	T->(E) T->id T->num	
20.	Write SDT for implementing a desk calculator and also construct annotated parse tree for the expression 3*5+4 into prefix. L->En E->E+T E->T T->T*F T->F F->(E) F->digit	8M
21.	Explain storage organization in a runtime environment with a neat diagram.	8M
22.	Explain activation trees and activation records with an example. (stack allocation)	8M
MODULE 5		
Code Generation		
1.	Explain the issues in the design of the code generator.	10M
2.	Briefly explain the simple code generator.	8M
3.	Explain register allocation and assignation using graph coloring method and usage the method with an example each.	10M
4.	Explain in detail peephole optimization techniques with examples.	10M
Machine Independent Optimization		
1.	Explain local optimization in detail, along with techniques and examples.	8M
2.	What is code optimization? Explain various levels and types of optimizations	10M
3.	What is the purpose of code optimization? Explain in detail loop optimization with example	10M
4.	Apply the code optimization techniques to the flow graph generated for quicksort.	10M
5.	Write the algorithm to generate basic blocks and flow graph for quick sort algorithm	10M
6.	Explain the DAG representation of the basic block in detail with an example.	10M
Introduction to data flow analysis		
1.	Define each of the following with an example. i) Reaching definition ii) Live variable iii) Available expression.	9M